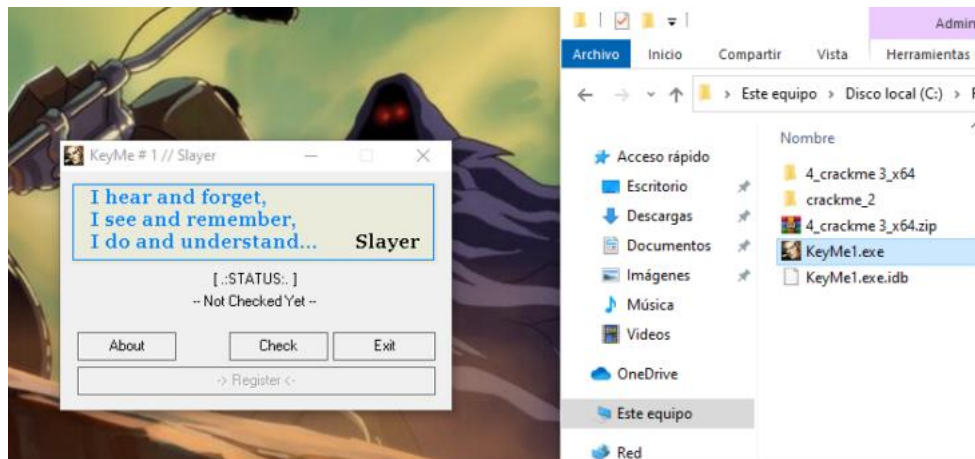




Target - KeyMe #1 – Slayer

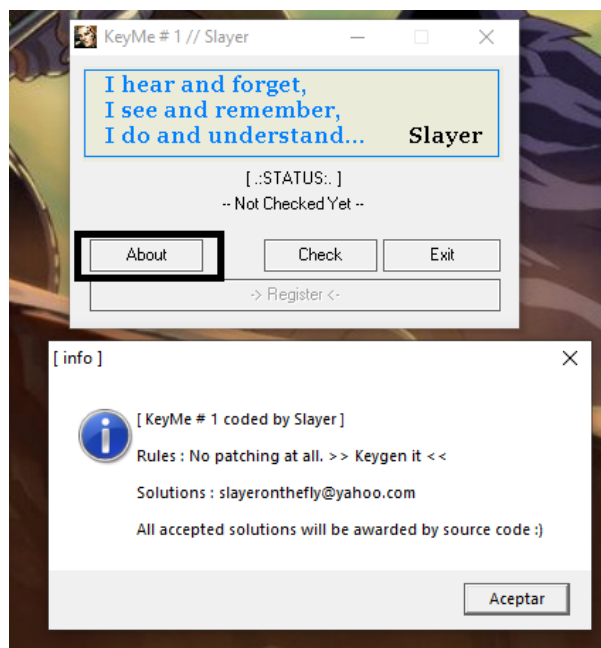
Eugenio D. Deluca (KarasuRØØT) – Date 19/09/2024

Hello again! Here I am, trying to keep learning a bit of reverse engineering. Alright, let's dive into the crackme "**KeyMe #1**" by author **Slayer**. Let's see what we are dealing with:



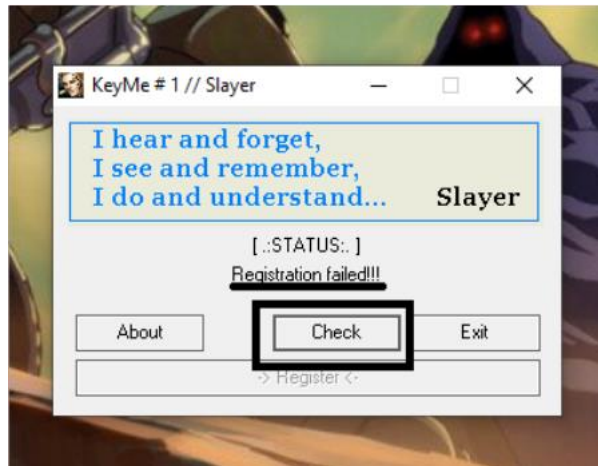
Upon executing it, a window opens with 3 available buttons (**About**, **Check**, **Exit**) and one grayed-out button (**Register**).

If I click on "**About**", it shows the following information:



Basically, it indicates that not everything can be solved by patching—we need to generate a Keygen and send the solution to the email address provided.

If I click on the "**Check**" button:



Oh, what a surprise—it tells us that the registration failed... how nice. And as expected, the **Exit** button closes the program.

I must confess that what the sign indicated caught my attention; actually, one of my hopes was that it would give me some kind of hint.

"I hear and forget, I see and remember, I do and understand"

Translated, it would be something like: "I hear and forget, I see and remember, I do and I understand"

Personally, I didn't find anything. That's why I'm moving on in a different direction.

If I open it in IDA to review it... to be honest, right now I don't know how to look at it. I will continue with x64dbg—

```

; File Name      : C:\Reversing\HerramientasREVERSING\Ejercicios-SOLID\KeyMe1.exe
; Format        : Portable executable for 80386 (PE)
; Imagebase     : 400000
; Timestamp     : 41F6DF88 (Wed Jan 26 00:08:43 2005)
; Section 1: (virtual address 00001000)
; Virtual size  : 00000388 ( 904.)
; Section size in file : 00000400 ( 1024.)
; Offset to raw data for section: 00000400
; Flags 60000020: Text Executable Readable
; Alignment    : default

.686p
.mmx
.model flat

; Segment type: Pure code
; Segment permissions: Read/Execute
_text segment para public 'CODE' use32
assume cs:_text
org 401000h
assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing

; Attributes: noreturn

public start
start proc near
push 0 ; lpModuleName
call GetModuleHandleA
mov hInstance, eax
push 0 ; dwInitParam
push offset DialogFunc ; lpDialogFunc
push 0 ; hInstParent
push 64h ; 'd' ; lpTemplate
push hInstance ; hInstance
call DialogBoxParamA
push 0 ; uExitCode
call ExitProcess
start endp

```

In principle, we can say that the little program starts there—the x64 architecture suggests that it is the entrypoint:

```

KeyMe1.exe - PID: 7668 - Module: keyme1.exe - Thread: Main Thread 2556 - x32dbg [Elevated]
File View Debug Tracing Plugins Favourites Options Help Jan 5 2022 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source

EIP ECX EDX ESI 00401000 6A 00 push 0
00401002 E8 21030000 call <JMP.&GetModuleHandleA>
00401007 A3 37324000 mov dword ptr ds:[403237],eax
0040100C 6A 00 push 0
0040100E 68 29104000 push 401029
00401013 6A 00 push 0
00401015 6A 64 push 64
00401017 FF35 37324000 push dword ptr ds:[403237]
0040101D E8 2A030000 call <JMP.&DialogBoxParamA>
00401022 6A 00 push 0
00401024 E8 ED020000 call <JMP.&ExitProcess>
00401029 55 push ebp
0040102A 8BEC mov ebp,esp
0040102C 817D 0C 10010000 cmp dword ptr ss:[ebp+C],110
00401033 0F85 95000000 jne 4010CE
00401039 68 E8030000 push 3E8
0040103E FF35 37324000 push dword ptr ds:[403237]
00401044 E8 21030000 call <JMP.&LoadIconA>
00401049 50 push eax
0040104A 6A 01 push 1
0040104C 68 80000000 push 80
00401051 FF75 08 push dword ptr ss:[ebp+8]
00401054 E8 23030000 call <JMP.&SendMessageA>
00401059 6A 67 push 67

.text:00401000 keyme1.exe:$1000 #400 <EntryPoint>

```

I try to search for what strings it has:

```

String
"[ info ]"
"reg.key"
"Memory allocation error!"
"Good work. You have done it!"
"Registration failed!!!"
"Step 1 ok -> now Register it!"
"Registration failed!!!"
L"t-ms-win-firewallapi-webproxy-11-1-1"
L"\\/"
L"win.ini"
L"Nation"
L"Control Panel\\International\\Geo"
L"\\Software\\Microsoft\\Windows NT\\CurrentVersion"
L"AppCompatFlags\\Layers"
L"\\Software\\Microsoft\\Windows\\CurrentVersion\\Explorer\\Shell Folders"
L"SIGN.MEDIA="

```

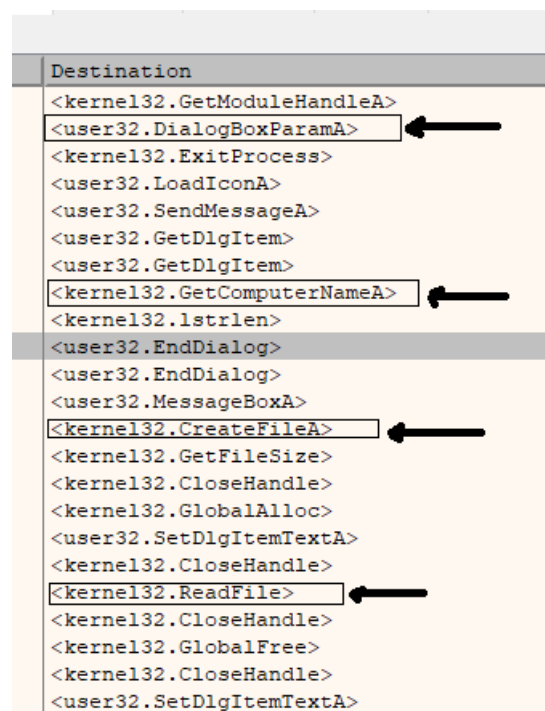
If I go to the disassembler, look around, and trace, I begin to see that there are several call instructions that call APIs.

```

004011E0 EB 4C jmp 40122E
004011E2 FF35 4A334000 push dword ptr ds:[40334A]
004011E8 E8 1D010000 call <JMP.&CloseHandle>
004011ED 56 push esi
004011EE 52 push edx
004011EF 8B35 4E334000 mov esi,dword ptr ds:[40334E]
004011F5 8B06 mov eax,dword ptr ds:[esi]
004011F7 83C6 04 add esi,4
004011FA 8B16 mov edx,dword ptr ds:[esi]
004011FC 33C2 xor eax,edx
004011FE 8B15 42334000 mov edx,dword ptr ds:[403342]
00401204 03C2 add eax,edx
00401206 5A pop edx
00401207 5E pop esi
00401208 3B05 3E334000 cmp eax,dword ptr ds:[40333E]
0040120E 75 1E jne 40122E
00401210 68 D9304000 push keyme1.4030D9
00401215 6A 66 push 66
00401217 FF75 08 push dword ptr ss:[ebp+8]
0040121A E8 63010000 call <JMP.&SetDlgItemTextA>
0040121F 6A 00 push 0
00401221 FF35 5A334000 push dword ptr ds:[40335A]
00401227 E8 26010000 call <JMP.&EnableWindow>
0040122C EB 29 jmp 401257
0040122E 68 F6304000 push keyme1.4030F6
00401233 6A 66 push 66

```

There are several APIs of interest; I won't list them all, but I will include what I consider to be the most important ones for this crackme:



GetComputerName: Retrieves only the NetBIOS name of the local computer.

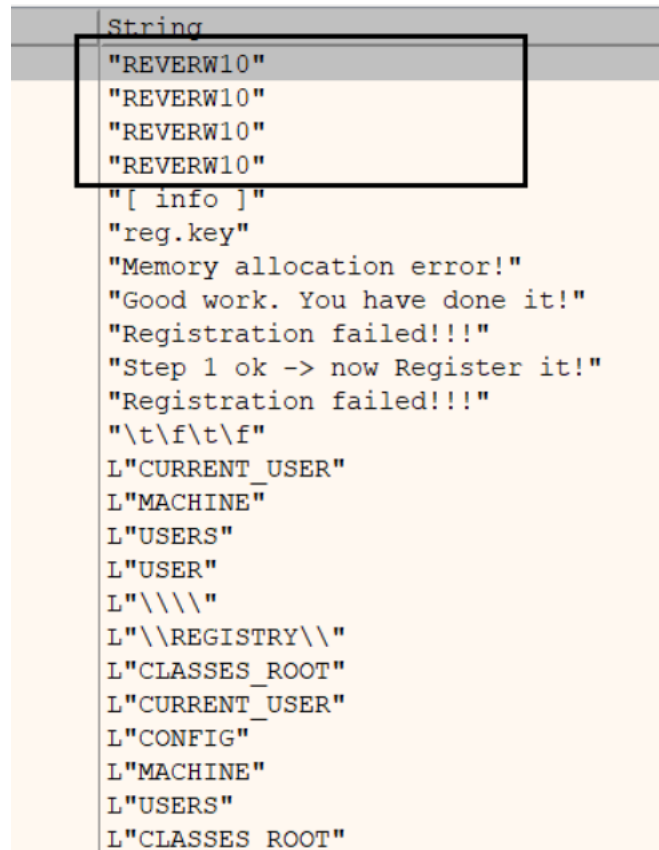
CreateFileA: Creates or opens a file or I/O device. The most commonly used I/O devices are the following: file, file stream, directory, physical disk, volume, console buffer, tape drive, communications resource, mailslot, and pipe.

ReadFile: Reads data from the specified file or input/output (I/O) device. Reads are performed at the position specified by the file pointer if the device supports it. This function is designed for both synchronous and asynchronous operations.

DialogBoxParam: The DialogBoxParam function uses the CreateWindowEx function to create the dialog box. Next, DialogBoxParam sends a WM_INITDIALOG message (and a WM_SETFONT message if the template specifies the DS_SETFONT or DS_SHELLFONT style) to the dialog box procedure. The function displays the dialog box (regardless of whether the template specifies the WS_VISIBLE style), disables the owner window, and starts its own message loop to retrieve and send messages for the dialog box. When the dialog box procedure calls the EndDialog function, DialogBoxParam destroys the dialog box, ends the message loop, and enables the owner window (if it was previously enabled) —aside from this, I think this is what disables the “Register” button for us.

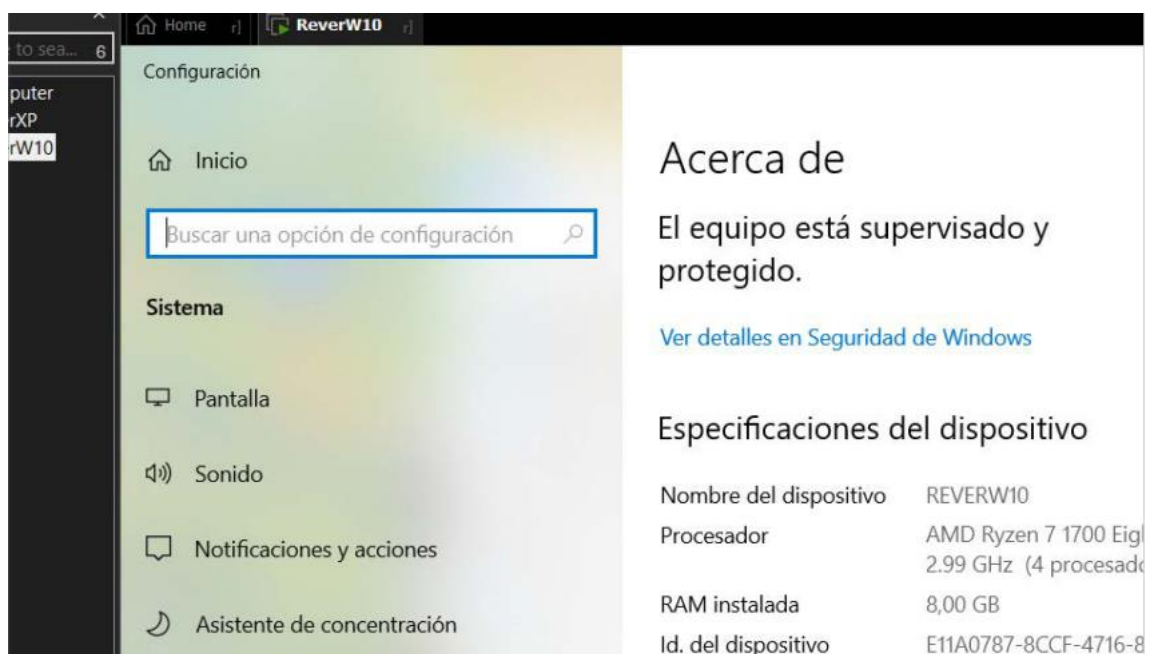
¹ Remember that all documentation regarding APIs can be found by searching Google for "msdn + ApiName" -> and the important things are: **What does it do? - What parameters does it use? - What does it return?**

What a curious thing cracking is. If we recall a few pages back, I left an image showing what the string search looked like, which contained several chains. But after analyzing the APIs and reviewing what each one did, it occurred to me to search for strings again *after* running the crackme with **F9**, and I found this:



```
String
"REVERW10"
"REVERW10"
"REVERW10"
"REVERW10"
"[ info ]"
"reg.key"
"Memory allocation error!"
"Good work. You have done it!"
"Registration failed!!!"
"Step 1 ok -> now Register it!"
"Registration failed!!!"
"\t\f\t\f"
L"CURRENT_USER"
L"MACHINE"
L"USERS"
L"USER"
L"\\\\"
L"\\REGISTRY\\"
L"CLASSES_ROOT"
L"CURRENT_USER"
L"CONFIG"
L"MACHINE"
L"USERS"
L"CLASSES_ROOT"
```

Remember the API **GetComputerName**: "Retrieves only the NetBIOS name of the local computer." -> and since that name is more or less how I usually name the VMs I create, it caught my attention:



Since I saw that, well, I wanted to know what it was doing with the hostname of the machine where the crackme runs. So, it occurred to me to set a breakpoint on the call that references the aforementioned API.

The screenshot shows a debugger window with the following assembly code:

```

00401096 68 3B324000 call <JMP.&GetComputerNameA>
0040109B E8 A0020000 push keym1.40323B
004010A0 A3 46334000 call <JMP.&strlen>
004010A5 59 mov dword ptr ds:[403346],eax
004010A6 58 pop ecx
004010A7 58 pop eax
004010A8 60 pushad
004010A9 9C pushfd
004010AA BE 3B324000 mov esi,keym1.40323B
004010AB C705 3E334000 00000000 mov dword ptr ds:[40333E],0
004010AC AC lodsb
004010AD 0FB6C0 movzx eax,al
004010AE 0105 3E334000 add dword ptr ds:[40333E],eax
004010B0 3C 00 cmp al,0
004010B1 75 F2 jne 4010B8
004010B2 90 popfd
004010B3 61 popad
004010B4 33C0 xor eax,eax
004010B5 C9 leave
004010B6 C2 1000 ret 10
004010B7 837D 0C 10 cmp dword ptr ss:[ebp+C],10
004010B9 75 0A jne 4010DE
004010BA 6A 00 push 0

```

The right pane shows the CPU registers with the following values:

```

EAX 00000000
EBX 00000000
ECX B8E665F
EDX 0040323 "REVERW10"
EBI 0019FAE
ESI 0019FAE
EDI 004F056
EIP 00000111

```

We see that after taking the hostname, there is the call <JMP.&strlen> -> According to what we understand, strlen "determines the length of the specified string (not including the terminating null character)".

Okay, I'll be honest, I started tracing using purely **F8** and got nowhere near anything but the **Bad Boy** alert, resigned to the idea that I was just "wasting time". I was already about to restart the process to see if I had missed something. However, this is where I stumbled upon something interesting while tracing the code:

The screenshot shows a debugger window with the following assembly code:

```

0040125D E8 D2000000 call <JMP.&GlobalFree>
00401262 66:83F8 67 cmp ax,67
00401266 0F85 98000000 jne 401304
0040126C 6A 00 push 0
0040126E E8 03010000 call <JMP.&OpenClipboard>
00401273 6A 01 push 1
00401275 E8 E4000000 call <JMP.&GetClipboardData>
0040127A 0BC0 or eax,eax
0040127C 74 72 je 4012F0
0040127E A3 33324000 mov dword ptr ds:[403233],eax
00401283 60 pushad

```

The right pane shows the CPU registers with the following values:

```

EAX 00000000
EBX 00000000
ECX B8E665F
EDX 0040323 "REVERW10"
EBI 0019FAE
ESI 0019FAE
EDI 004F056
EIP 00000111

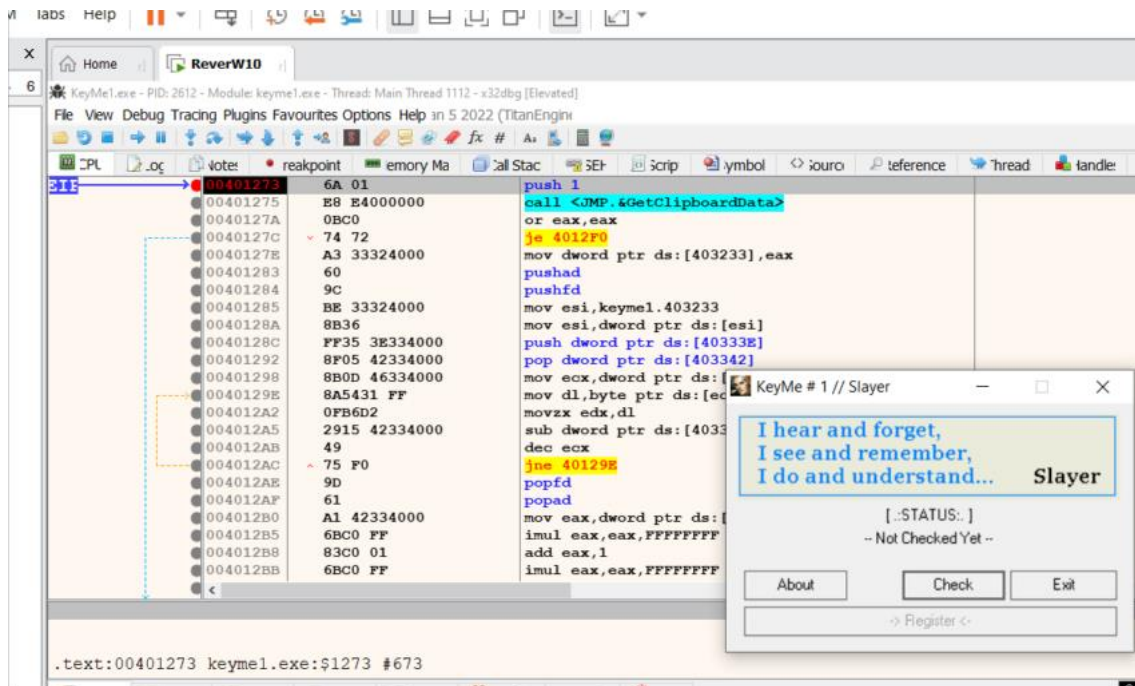
```

OpenClipboard: Opens the clipboard for examination and prevents other applications from modifying the content of the clipboard.

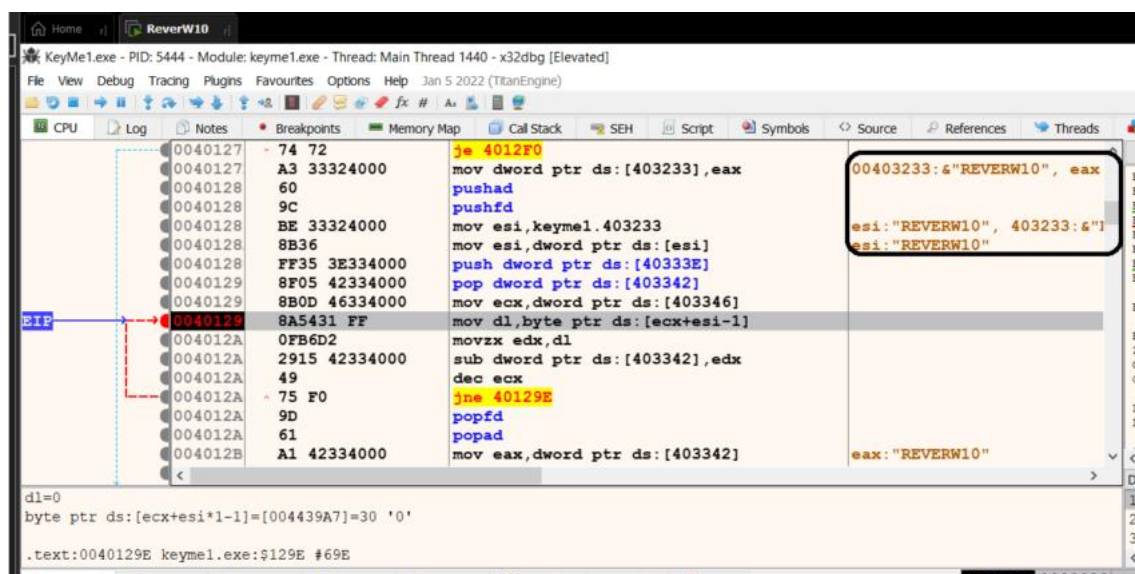
GetClipboardData: Retrieves data from the clipboard in a specified format. The clipboard must have been opened previously.

This made me think that I needed to have something in the clipboard, and then the Get function retrieves whatever is there. So, I'm going to "cheat" a little: I will copy my hostname "**ReverW10**" and set a breakpoint on the first call where it opens the clipboard to see what happens. I restart and run it with **F9**.

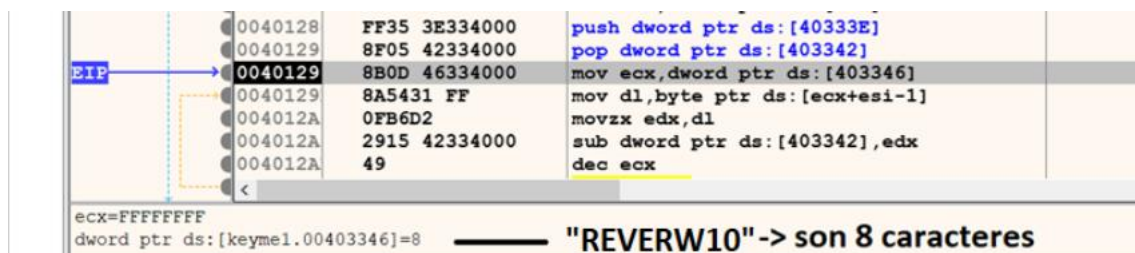
Since it wasn't doing anything, I selected the "Check" button and it stopped right here:

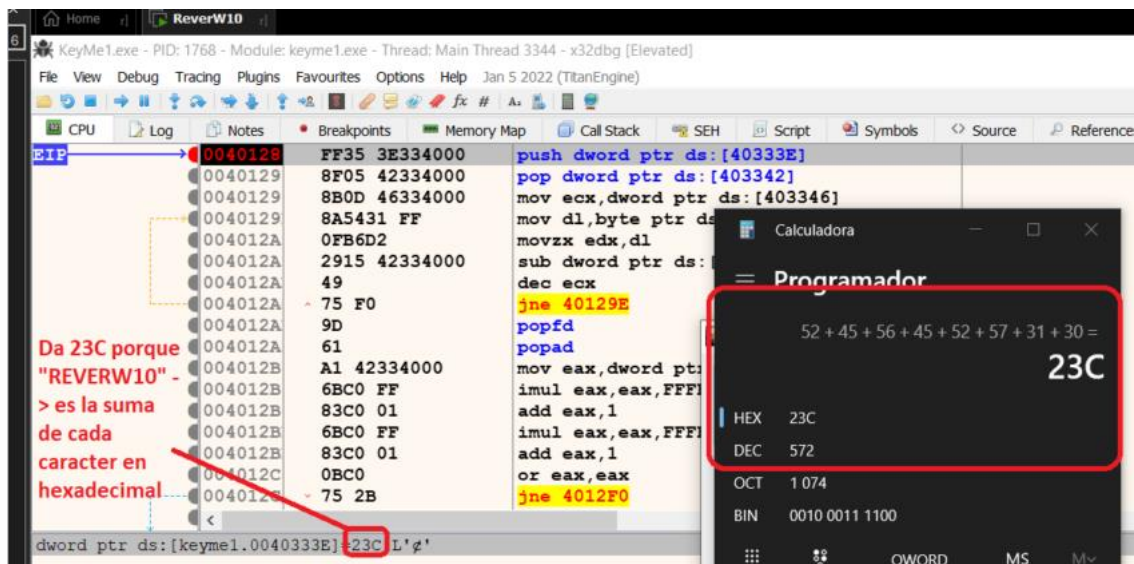


After that, it begins to go back and forth and take a ton of loops, so I will try to simplify it a bit (Note: yes, I had to do this several times because I kept getting lost).

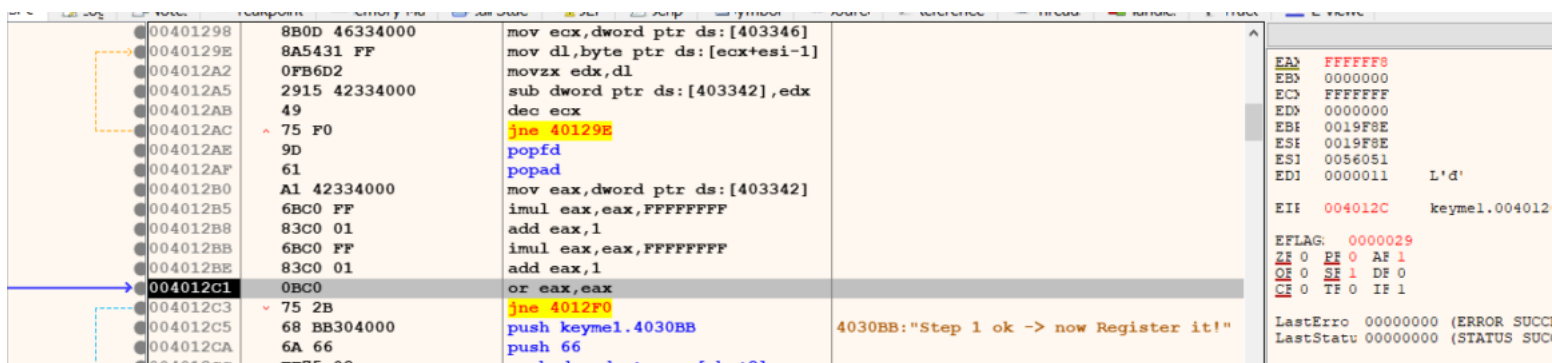


Here, for example, it enters a loop to iterate through the string "REVERW10" and I see that it performs several operations.





Therefore, what is stored at `[40333E]` would be the sum of the hexadecimal characters of the hostname. And that's it...



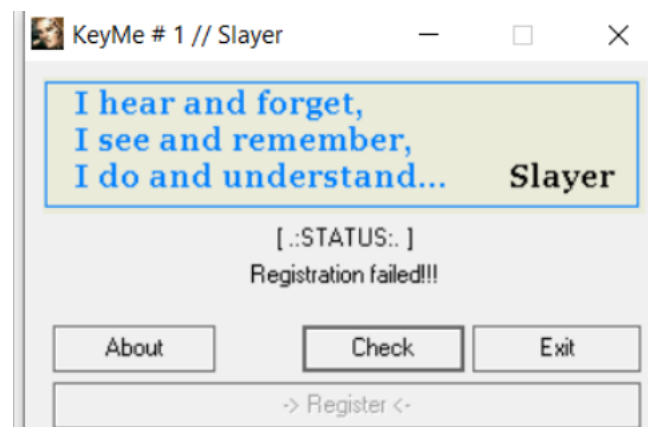
Since this took a lot of time and... there are things that go beyond just copying and pasting an image, you will have to trust me or test it for yourselves.

Alright, after going back and forth so much and reviewing my notes multiple times, during a brief moment of clarity, I began to think that somewhere it was using the `GetClipboardData` API, or at least it was a strong possibility. So, I decided to test it by copying my hostname into the clipboard and clicking "Check".



Since my hostname is **REVERW10** (without quotes), I copied it into a notepad. Then, I opened Slayer's KeyMe #1 -> I clicked the "Check" button and it enabled the ->Register <- button!

Great, one step further and we haven't broken the rule that "prevents" us from patching the program. The happiness lasted only a moment—just the time it took to click "Register" and realize that we are right back at the beginning... or something like that.



It's here where I had no other choice but to keep tracing, and I found the following:

00401112	6A 03	push 3	
00401112	6A 00	push 0	
00401112	6A 01	push 1	
00401112	68 00000080	push 80000000	
00401113	68 26314000	push keyme1.403126	403126: "reg.key"
00401113	E8 D3010000	call <JMP.&CreateFileA>	
00401113	83F8 FF	cmp eax,FFFFFFFF	
00401114	75 05	jne 401147	

I had already seen the `CreateFileA` API—but to keep it handy, what it basically does is: It creates or opens a file or an I/O device. The most frequently used I/O devices are the following: files, file streams, directories, etc.

Alright, but creating files, at least this KeyMe, doesn't do it, so I initially decided to create it myself. And why not? So I went ahead and created a `reg.key` file in the same location as `KeyME1.exe` (on the Desktop). As for the content of that file, it occurred to me to use the same name as the PC—in my case, "REVERW10"—but keeping in mind to have that same name in the clipboard as well. I manipulated `reg.key` with Notepad at first. Now then, after reviewing, we reached this line:

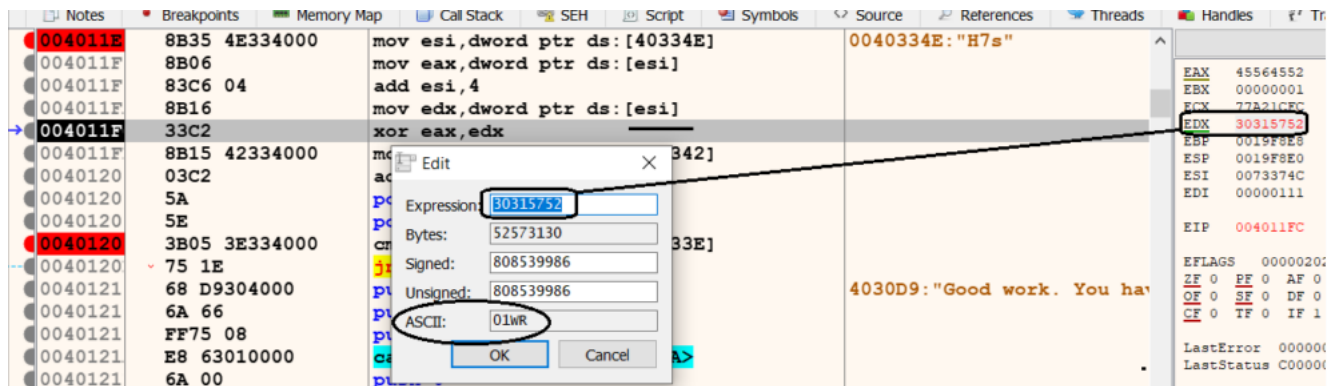
4011F	8B06	mov eax,dword ptr ds:[esi]	
4011F	83C6 04	add esi,4	
4011F	8B16	mov edx,dword ptr ds:[esi]	
4011F	33C2	xor eax,edx	
4011F	8B15 42334000	mov edx,dword ptr ds:[403342]	
40120	03C2	add eax,edx	
40120	5A	pop edx	
40120	5E	pop esi	
40120	3B05 3E334000	cmp eax,dword ptr ds:[40333E]	
40120	75 1E	jne 40122E	
40121	68 D9304000	push keyme1.4030D9	
40121	6A 66	push 66	

Expression:	45564552
Bytes:	52455645
Signed:	1163281746
Unsigned:	1163281746
ASCII:	EVER

EAX	45564552
EBX	00000001
ECX	77A21CFC
EDX	00000000
EBP	0019F8E8
ESP	0019F8E0
ESI	00733748
EDI	00000111
EIP	004011F7
EFLAGS	00000246
ZF	1
PF	1
AF	0
OF	0
SF	0
DF	0
CF	0
TF	0
IF	1

I noticed that in `EAX` there was the value 45564552—which in ASCII, as indicated, is represented as **EVER**.

But in reality, if we remember, here it displays it as "**Little Endian**" -> so what we are actually seeing there is "**REVE**". And later I was also able to verify it in `EDX`, which has the value 30315752, representing "**01WR**" in ASCII—also in Little Endian.

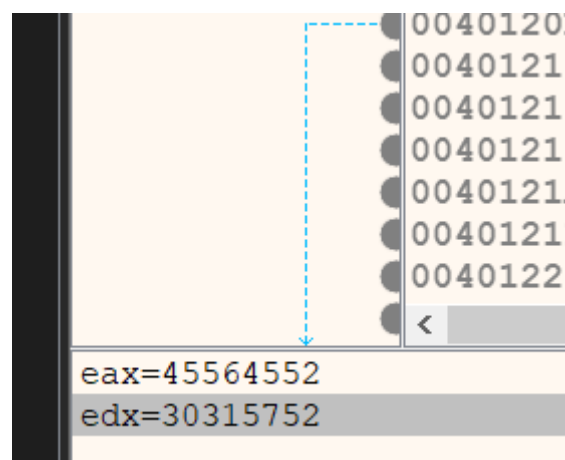


Here is the translation:

That is to say, it basically takes the first 4 characters of my PC's name and moves them to `EAX` in Little Endian, as mentioned before. Then, it takes the next 4 characters as well and puts them into `EDX`. If we look closely at the image, it then performs a `XOR EAX, EDX`.

This was incredibly difficult for me to understand, which is why I want to take this opportunity to thank that kind soul who helped me wrap my head around it, hehe—"The most important thing is always to believe in yourself, but a little help from others is a great blessing." To make it a bit clearer, let's follow the example of my PC. The name is **REVERW10**. Alright, what it does is the following: it passes "EVER" (which is **REVE**) to `EAX`, and it passes "01WR" (which would be **RW10**) to `EDX`.

BUT PAY ATTENTION: when it executes `XOR EAX, EDX`, it does so using the corresponding values currently held in `EAX` and `EDX`—meaning `EAX` = 45564552 and `EDX` = 30315752. In fact, we can see this a little further down in the disassembler's hint section:



Here we see the result, and it leaves it in EAX

The screenshot shows a debugger window with the following assembly code:

```
mov esi,dword ptr ds:[40334E]
mov eax,dword ptr ds:[esi]
add esi,4
mov edx,dword ptr ds:[esi]
xor eax,edx
mov edx,dword ptr ds:[403342]
add eax,edx
pop edx
pop esi
cmp eax,dword ptr ds:[40333E]
jne 40122E
push keyme1.4030D9
push 66
push dword ptr ss:[ebp+8]
call <JMP.&SetDlgItemTextA>
push 0
```

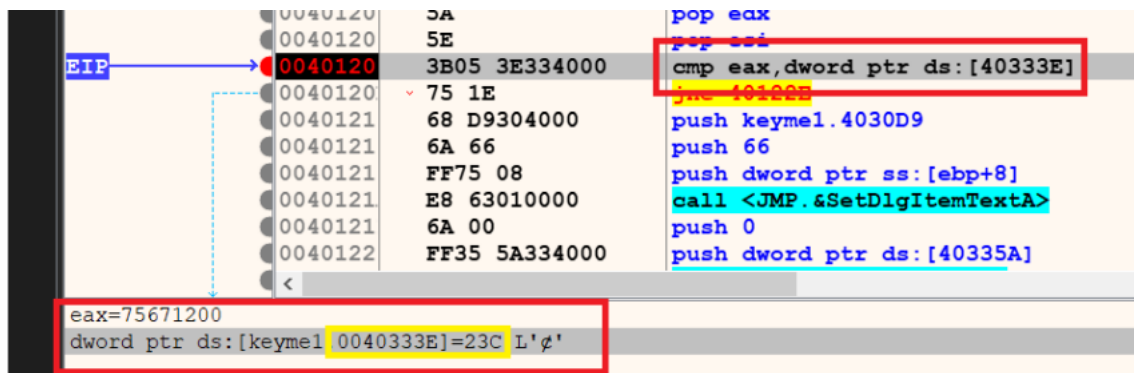
The register list on the right shows the following values:

Register	Value
EAX	75671200
EBX	00000001
ECX	77A21CFC
EDX	30315752
EBP	0019F8E8
ESP	0019F8E0
ESI	0073374C

A calculator window titled "Calculadora" is overlaid on the bottom right, showing the calculation:

$$45564552 \text{ XOR } 30315752 = 7567 \ 1200$$

In this line, I see that `EAX` now holds the XORed result, and what is contained in `[40333E]` is `23C`—which, if we remember, `23C` is the result of adding up each character of my hostname “REVERW10” in HEX.



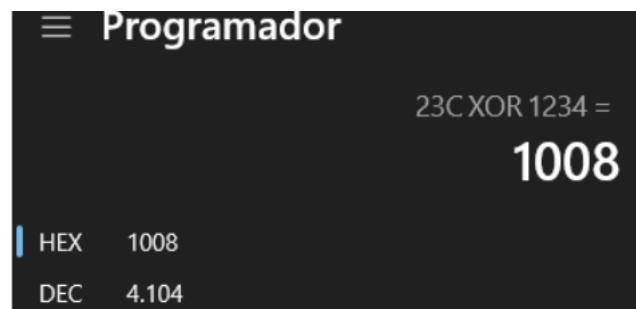
Therefore, what it compares is basically what the image indicates—it compares the XORed result of EAX and EDX (which, let's remember, take the first 4 bytes in one and the next 4 in the other, at least in my example) against the sum of each character of my PC's name in hexadecimal.

This means that in `reg.key`, I need to have the values that, when XORed together, equal 23C in my case. Thanks to Solid for pointing out the following to me, maybe it will be useful to others—if we think of it as $A \oplus B = \text{23C}$, A and B are my unknowns that yield 23C after a XOR operation.

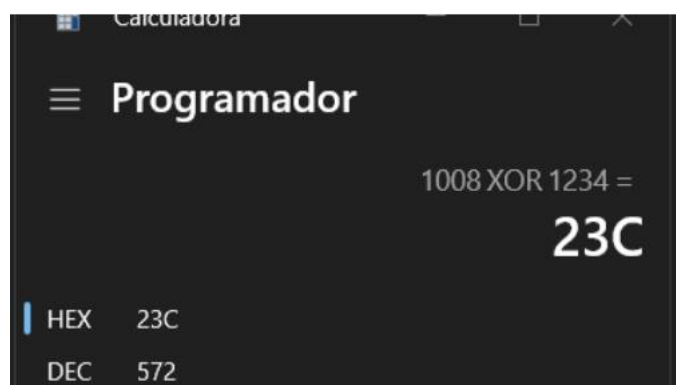
"If $A \oplus B = C$ implies $C \oplus B = A$ implies $A \oplus C = B$ "

And here comes the major detail that blew my mind, ha. The thing is, I'm not very good at math and such. But technically, in our situation with the crackme, we actually have only one unknown.

But how? Why? Well... I understood it like this: $C = \text{23C}$, and either A or B must be a constant or a fixed number. For example, I did it this way:



That means that 1008 in Hex, or 4104 in Decimal, would be our unknown—why? Because of the formula from before. If I do: 1008 XOR 1234, it should give 23C—just as the calculator shows us



We can also test it using the Python console.

```
Símbolo del sistema - python
Microsoft Windows [Versión 10.0.19045.4894]
(c) Microsoft Corporation. Todos los derechos reservados

C:\Users\Home>python
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023)
Type "help", "copyright", "credits" or "license()" for more
>>> 10 ^ 20
30
>>> 30 ^ 20
10
>>> 10 ^ 30
20
>>>
```

```
Símbolo del sistema - python
Microsoft Windows [Versión 10.0.19045.4894]
(c) Microsoft Corporation. Todos los derechos reservados

C:\Users\Home>python
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023)
Type "help", "copyright", "credits" or "license()" for more
>>> 10 ^ 20
30
>>> 30 ^ 20
10
>>> 10 ^ 30
20
>>> 0x23c ^ 1234
1774
>>> 1774 ^ 1234
572
>>> hex(572)
'0x23c'
>>>
```

Let's review more or less what we have come up with so far:

- 1. To activate the "Register" button, we need to have the PC's hostname in the clipboard.
- 2. Next, we must have the `reg.key` file in the same directory as `KeyMe1.exe`. The value inside it must not exceed 8 bytes (4 bytes for the first part and 4 bytes for the second). Ah, but here is a catch: it needs to be in **Little Endian**.

To solve it:

Alright, let's follow the advice of the wise. Let's try to do it "on paper" first, and then we will see how to write the code in Python.

Following my example: my hostname is **REVERW10**.

So, if we understood correctly, traced everything, and did it all right—first, we need to make sure that name is in the clipboard. Alright, that's done.

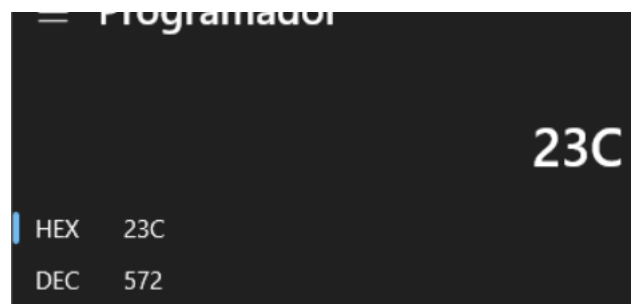


Alright, now if we remember what was said earlier, we know that **REVERW10 = 23C** in Hex—meaning that when my `reg.key` performs the operations, it must equal 23C.

That is to say, $23C \text{ XOR } 1234 = 1008$ (Hex) or 4104 (Decimal)—I'm choosing 1234 just as an arbitrary number (obviously, when you do the math, you should use your corresponding numbers).

Actually, there is a small catch. When I initially did it with the calculator, I took 23C (HEX) XOR 1234 (HEX), but it didn't give me the correct values per se—even though if I XOR the result again it brings me back to 23C. In reality, the calculation should be done like this:

23C -> In decimal is 572

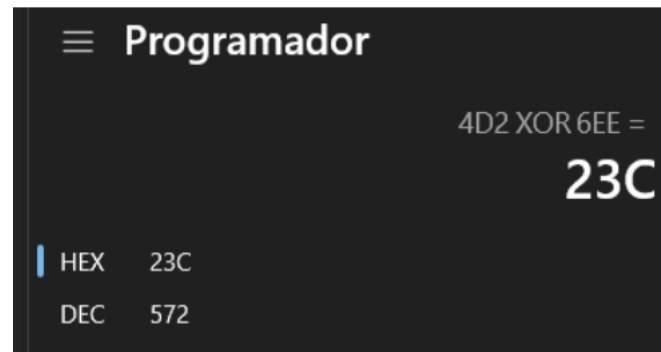


And 1234 in decimal—meaning I did 572 XOR 1234



That means it gives us **6EE** in Hex or **1774** in Decimal—Great!

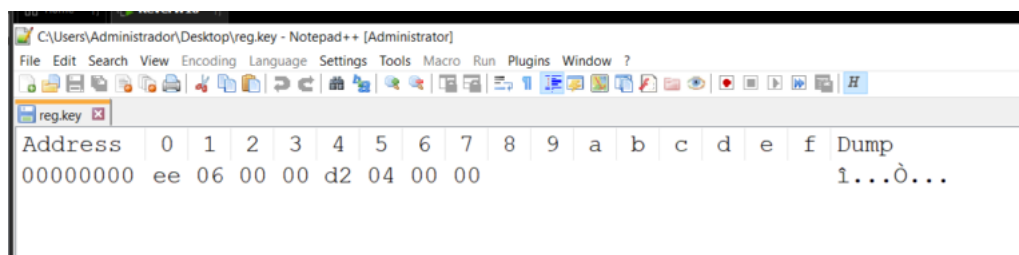
So, basically, if I do **6EE XOR 4D2**, it should give **23C**.



4D2 (hex) = 1234 in decimal.

Alright, then I ran into another problem—we are supposed to make it so that the corresponding value of **4D26EE** is saved in Notepad in ASCII format and in Little Endian. To do this, we will also need to pad the missing bytes.

With the help of Notepad++, it would look like this:



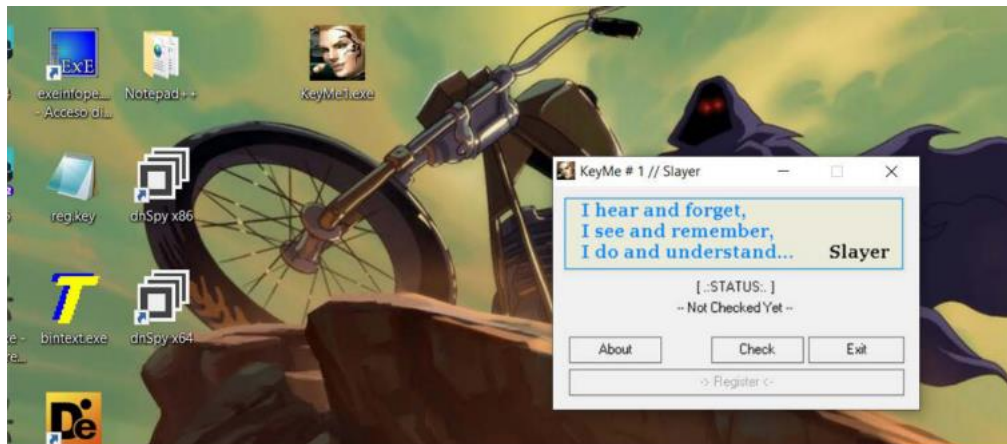
Here is the translation:

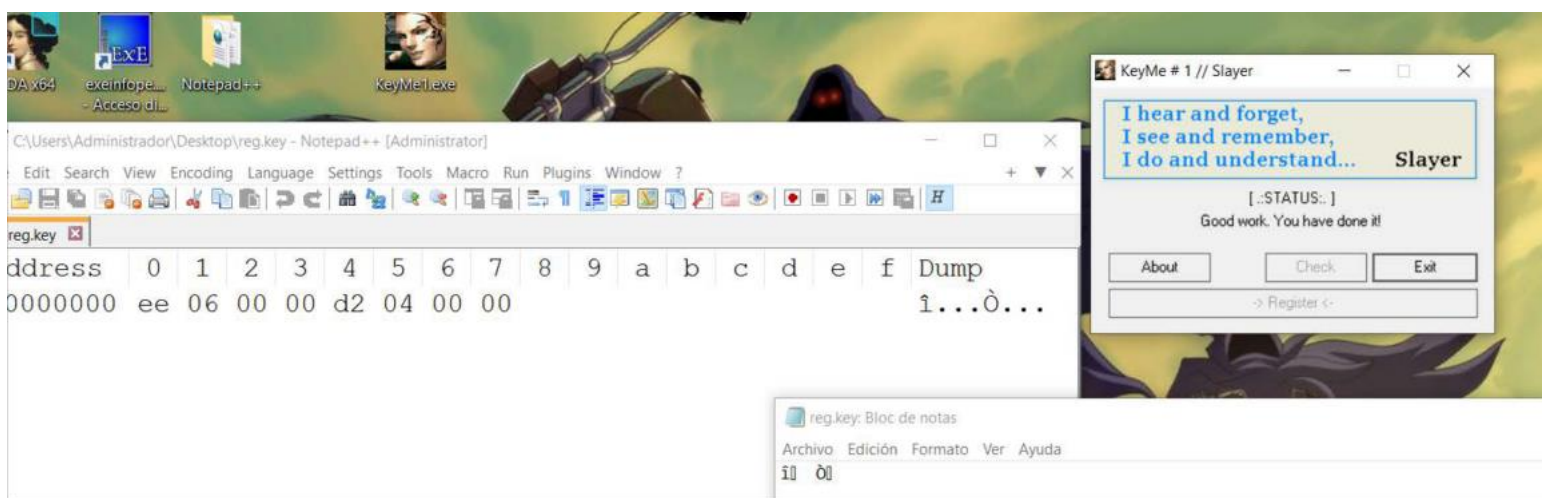
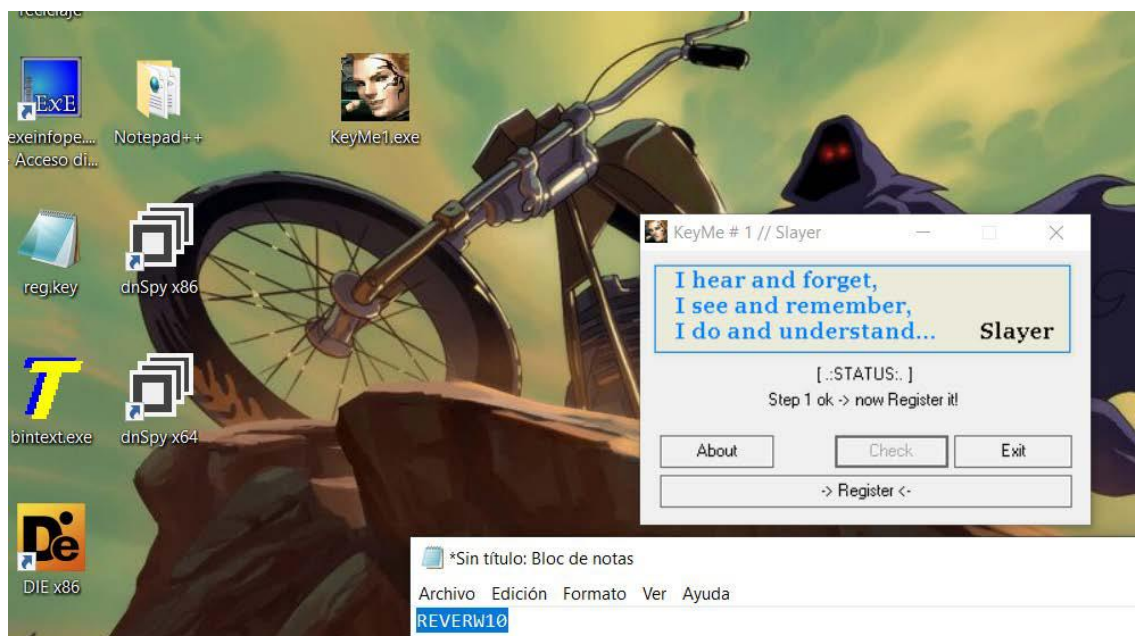
That is to say, **EE060000D2040000** (This is how it is represented in ASCII, padding the 8 bytes that the program evaluates somewhere).

If I remember correctly, it was right around here:

0040114	6A 00	push 0
0040114	FF35 4A334000	push dword ptr ds:[40334A]
0040115	E8 C9010000	call <JMP.&GetFileSize>
0040115	83F8 08	cmp eax,8
0040115	73 10	jae 40116E
0040115	FF35 4A334000	push dword ptr ds:[40334A]
0040116	E8 A1010000	call <JMP.&CloseHandle>

Alright then—if I were to use that reg.key with that hostname and run Slayer's KeyMe, it should work. Let's see:





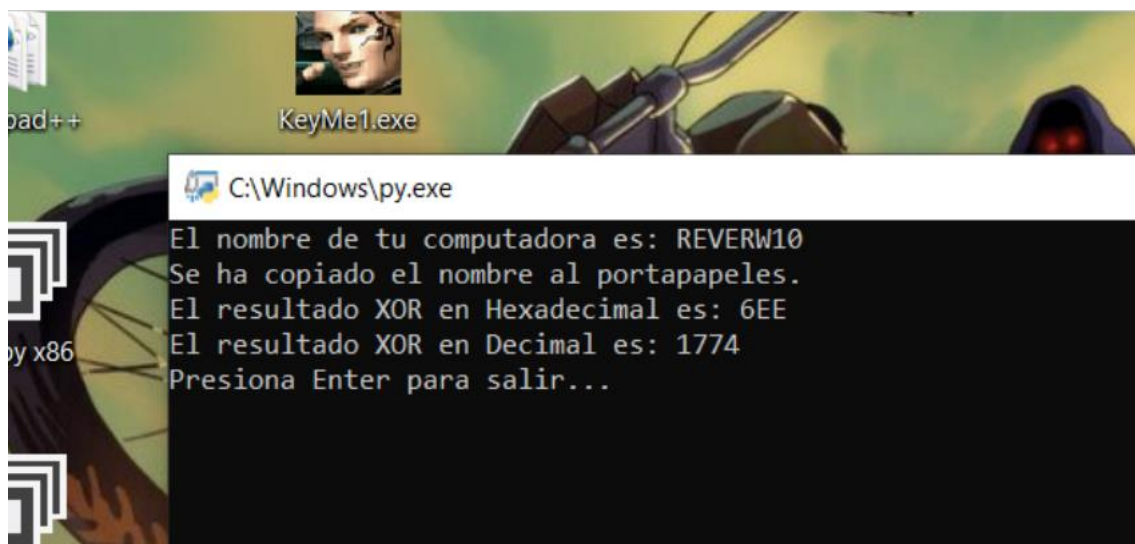
Awesome! We did it :D Note that in regular Notepad it looks the exact same as in Notepad++, but the latter, using the HEX EDITOR plugin -> View in HEX, shows us what it would look like in hexadecimal.

Finally, here is what a script in Python would look like:

```
import socket
import subprocess
import time
import os
import struct

nombre_pc = socket.gethostname()#Tomo el nombre de la PC
constante_xor = 1234 #Defino una constante para luego operar
contador = 0
# Calculo la suma hexadecimal directamente
for caracter in nombre_pc:
    valor_decimal = ord(caracter)
    valor_hexadecimal = hex(valor_decimal)
    contador += valor_decimal
sumaHEX = contador ^ constante_xor #Aca realizo la operacion de hacer la CONSTANTE XOR el resultado
estru0 = struct.pack("<I",constante_xor) #Converto datos de Python en una cadena de bytes, siguiendo un formato específico.
estru1 = struct.pack("<I", sumaHEX) #<: Indica que se usará el orden de bytes "little-endian"
final = estru1 + estru0 #Sumo las dos variables donde converti cada cadena de bytes
# Copiamos solo el nombre de la PC al portapapeles
subprocess.run(["clip"], input=nombre_pc.encode(), check=True)
# Imprimimos los resultados por consola
print("El nombre de tu computadora es:", nombre_pc)
print("Se ha copiado el nombre al portapapeles.")
print(f"El resultado XOR en Hexadecimal es: {sumaHEX:X}")
print(f"El resultado XOR en Decimal es: {sumaHEX}")
# Creo el archivo
ruta_escritorio = os.path.join(os.environ['USERPROFILE'], 'Desktop')
ruta_archivo = os.path.join(ruta_escritorio, "reg.key")
with open(ruta_archivo, "wb") as archivo:
    archivo.write(final)
# Mantener la consola abierta hasta que el usuario presione Enter
input("Presiona Enter para salir...")
```

The result of how it would look if I run it on the host named “REVERW10”:



Clarifications:

- To be honest, this crackme... personally, I found it quite difficult, but with a bit of help, patience, and some thinking, it can be done. Plus, it's really cool because of the neat details it has, like the APIs it uses.
- This works smoothly on 64-bit PCs (I don't have a 32-bit VM on hand).
- Everything goes well if the name is in uppercase, but if the hostname were *ReverW10*, for example—when it comes to generating the `reg.key` file, it generates everything fine, but when you click "Check," it won't validate it. This is due to a mistake I made when I wrote the Python code.

How did I fix it? Simple—by changing the line where it grabs the hostname so that it takes it in uppercase.

```
import socket
import subprocess
import time
import os
import struct

nombre_pc = socket.gethostname().upper()#Tomo el nombre de la PC en mayus
constante_xor = 1234 #Defino una constante para luego operar
contador = 0
for caracter in nombre_pc: # Calculo la suma hexadecimal directamente
    valor_decimal = ord(caracter)
    valor_hexadecimal = hex(valor_decimal)
    contador += valor_decimal
sumaHEX = contador ^ constante_xor #Aca realizo la operacion de hacer la CONSTANTE XOR el resultado
estru0 = struct.pack("<I",constante_xor) #Converto datos de Python en una cadena de bytes, siguiendo un formato específico.
estru1 = struct.pack("<I", sumaHEX) #<: Indica que se usará el orden de bytes "little-endian"
final = estru1 + estru0 #Sumo las dos variables donde converti cada cadena de bytes
subprocess.run(["clip"], input=nombre_pc.encode(), check=True)# Copiamos solo el nombre de la PC al portapapeles
print("El nombre de tu computadora es:", nombre_pc)# Imprimimos los resultados por consola
print("Se ha copiado el nombre al portapapeles.")
print(f"El resultado XOR en Hexadecimal es: {sumaHEX:X}")
print(f"El resultado XOR en Decimal es: {sumaHEX}")
ruta_escritorio = os.path.join(os.environ['USERPROFILE'], 'Desktop')# Creo el archivo
ruta_archivo = os.path.join(ruta_escritorio, "reg.key")
with open(ruta_archivo, "wb") as archivo:
    archivo.write(final)
# Mantener la consola abierta hasta que el usuario presione Enter
input("Presiona Enter para salir...")
```

Toma ahora el nombre
en Mayusculas